Tutorial: Đăng ký và đăng nhập với Spring Security 7 + Spring Boot 4.1 + Spring Data JPA

Oct 16, 2026 · @Ng Tuân

1. Tổng quan

Tutorial này xây dựng một ứng dụng web có 4 chức năng: đăng ký tài khoản, đăng nhập, đăng xuất và trang chào mừng chỉ dành cho người đã đăng nhập. Tài khoản lưu trong MySQL, mật khẩu băm bằng BCrypt, giao diện dùng Thymeleaf + Bootstrap.

Các kỹ thuật được áp dụng:

- Spring Security 7 với `SecurityFilterChain` theo lambda DSL, form login tùy biến, CSRF bật mặc định.
- `UserDetailsService` đọc tài khoản và vai trò (`ROLE_USER` , `ROLE_ADMIN`) từ database qua Spring Data JPA.
- Mật khẩu lưu bằng `DelegatingPasswordEncoder` (định dạng `{bcrypt}...`), dễ nâng cấp thuật toán về sau.
- Validation form đăng ký bằng Bean Validation, kể cả ràng buộc cấp lớp "mật khẩu nhập lại phải khớp"; thông báo lỗi tiếng Việt qua `messages.properties` .
- Tự đăng nhập ngay sau khi đăng ký thành công, lưu `SecurityContext` vào session đúng cách.
- Schema quản lý bằng Flyway; MySQL tự khởi động qua Docker Compose support.
- Test luồng đăng ký, đăng nhập, phân quyền bằng `spring-security-test` .

Kết quả cuối cùng tại `http://localhost:8080` :

URL	Truy cập	Chức năng
`/registration`	Mọi người	Form đăng ký tài khoản
`/login`	Mọi người	Form đăng nhập
`/`	Đã đăng nhập	Trang chào mừng, nút đăng xuất
`/admin/**`	Vai trò ADMIN	Khu vực quản trị (dùng cho bài tập)

2. Yêu cầu môi trường

Thành phần	Phiên bản	Ghi chú
JDK	25 (LTS)	Spring Boot 4.1 yêu cầu tối thiểu Java 17
Spring Boot	4.1.1	Quản lý phiên bản cho mọi thư viện bên dưới
Spring Security	7.1.x	Đi kèm Spring Boot 4.1
Thymeleaf + `thymeleaf-extras-springsecurity6`	Do Boot quản lý	Module `springsecurity6` vẫn là bản dùng cho Spring Security 7
Flyway	Do Boot quản lý	Cần thêm `flyway-mysql` cho MySQL
MySQL	8.4 LTS	Chạy trong container Docker
Maven	3.9+	Dùng Maven Wrapper đi kèm project
Docker	Docker Desktop hoặc Docker Engine + Compose v2	Để Spring Boot tự khởi động MySQL

Khi tạo project, chọn bản patch mới nhất của 4.1.x trên start.spring.io.

3. Khởi tạo project

Tạo project trên start.spring.io: Maven, Java 25, Spring Boot 4.1.x, group `com.fptaptech`, artifact `auth`. Chọn dependency: Spring Web, Spring Security, Thymeleaf, Spring Data JPA, Validation, Flyway Migration, MySQL Driver, Docker Compose Support. Các dependency còn lại thêm tay ở bước 4.

Cấu trúc thư mục đích:

```
auth/
├── compose.yaml
├── pom.xml
└── src/
```

```
    ├── main/
    │   ├── java/com/fptaptech/auth/
    │   │   ├── AuthApplication.java
    │   │   ├── config/
    │   │   │   └── SecurityConfig.java            # SecurityFilterChain,
PasswordEncoder
    │   │   ├── security/
    │   │   │   ├── DatabaseUserDetailsService.java
    │   │   │   └── AutoLogin.java                 # đăng nhập ngay sau khi
đăng ký
    │   │   ├── user/
    │   │   │   ├── AppUser.java                   # entity bảng users
    │   │   │   ├── Role.java                      # entity bảng roles
    │   │   │   ├── UserRepository.java
    │   │   │   ├── RoleRepository.java
    │   │   │   ├── UserService.java
    │   │   │   └── UsernameAlreadyExistsException.java
    │   │   └── web/
    │   │       ├── AuthController.java
    │   │       ├── RegistrationForm.java
    │   │       ├── PasswordsMatch.java            # ràng buộc cấp lớp
    │   │       └── PasswordsMatchValidator.java
    │   └── resources/
    │       ├── application.yaml
    │       ├── messages.properties                # thông báo lỗi tiếng Việt
    │       ├── db/migration/
    │       │   └── V1__init_schema.sql
    │       └── templates/
    │           ├── fragments.html                 # phần <head> dùng chung
    │           ├── login.html
    │           ├── registration.html
    │           └── welcome.html
    └── test/
        ├── java/com/fptaptech/auth/
        │   └── AuthFlowTest.java
        └── resources/
            └── application-test.yaml
```

4. pom.xml

Spring Boot 4 đã module hóa starter: Flyway cần `spring-boot-starter-flyway` (chỉ thêm `flyway-core` thì auto-configuration không kích hoạt), test bảo mật dùng `spring-boot-starter-security-test`.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.1</version>
        <relativePath/>
    </parent>

    <groupId>com.fptaptech</groupId>
    <artifactId>auth</artifactId>
    <version>1.0.0</version>

    <properties>
        <java.version>25</java.version>
    </properties>

    <dependencies>
        <!-- Web + View -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
```

```
        <!-- Security -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.thymeleaf.extras</groupId>
            <artifactId>thymeleaf-extras-springsecurity6</artifactId>
        </dependency>

        <!-- Data -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-flyway</artifactId>
        </dependency>
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-mysql</artifactId>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-docker-compose</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <!-- Test -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
```

```
            <artifactId>spring-boot-starter-security-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

5. Mô hình dữ liệu

Một người dùng có nhiều vai trò, một vai trò gán cho nhiều người dùng: quan hệ nhiều-nhiều qua bảng `users_roles`. Schema do Flyway tạo; Hibernate chỉ kiểm tra (`ddl-auto: validate`).

5.1 Migration `V1__init_schema.sql`

`src/main/resources/db/migration/V1__init_schema.sql`:

```
CREATE TABLE roles (
    id   BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE users (
    id            BIGINT AUTO_INCREMENT PRIMARY KEY,
    username      VARCHAR(50)  NOT NULL UNIQUE,
    password_hash VARCHAR(100) NOT NULL,
    enabled       BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at    DATETIME(6)  NOT NULL
);
```

```
CREATE TABLE users_roles (
    user_id BIGINT NOT NULL,
    role_id BIGINT NOT NULL,
    PRIMARY KEY (user_id, role_id),
    CONSTRAINT fk_users_roles_user FOREIGN KEY (user_id) REFERENCES users
(id) ON DELETE CASCADE,
    CONSTRAINT fk_users_roles_role FOREIGN KEY (role_id) REFERENCES roles
(id)
);

INSERT INTO roles (name) VALUES ('ROLE_USER'), ('ROLE_ADMIN');
```

`password_hash` dài 100 ký tự: chuỗi BCrypt 60 ký tự cộng tiền tố `{bcrypt}`, còn dư chỗ khi đổi thuật toán.

5.2 Entity

```
package com.fptaptech.auth.user;

import jakarta.persistence.*;

@Entity
@Table(name = "roles")
public class Role {

    public static final String USER = "ROLE_USER";
    public static final String ADMIN = "ROLE_ADMIN";

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String name;

    protected Role() {}

    public Long getId() { return id; }
    public String getName() { return name; }
}
```

```
package com.fptaptech.auth.user;

import jakarta.persistence.*;

import java.time.Instant;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "users")
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(name = "password_hash", nullable = false, length = 100)
    private String passwordHash;

    @Column(nullable = false)
    private boolean enabled = true;

    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(name = "users_roles",
            joinColumns = @JoinColumn(name = "user_id"),
            inverseJoinColumns = @JoinColumn(name = "role_id"))
    private Set<Role> roles = new HashSet<>();

    protected AppUser() {}

    public AppUser(String username, String passwordHash) {
        this.username = username;
        this.passwordHash = passwordHash;
    }

    @PrePersist
    void onCreate() {
```

```
        this.createdAt = Instant.now();
    }

    public Long getId() { return id; }
    public String getUsername() { return username; }
    public String getPasswordHash() { return passwordHash; }
    public boolean isEnabled() { return enabled; }
    public Instant getCreatedAt() { return createdAt; }
    public Set<Role> getRoles() { return roles; }
}
```

Entity đặt tên `AppUser` để không trùng với `org.springframework.security.core.userdetails.User` dùng ở mục 6.

5.3 Repository

`@EntityGraph` nạp luôn `roles` trong cùng câu truy vấn khi đăng nhập, tránh `LazyInitializationException` và N+1.

```
package com.fptaptech.auth.user;

import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<AppUser, Long> {

    @EntityGraph(attributePaths = "roles")
    Optional<AppUser> findByUsername(String username);

    boolean existsByUsername(String username);
}
```

```
package com.fptaptech.auth.user;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface RoleRepository extends JpaRepository<Role, Long> {

    Optional<Role> findByName(String name);
}
```

6. Tầng service

Ba thành phần: `UserService` tạo tài khoản, `DatabaseUserDetailsService` cho Spring Security đọc tài khoản khi đăng nhập, `AutoLogin` đăng nhập hộ người dùng ngay sau khi đăng ký.

6.1 Đăng ký tài khoản

Username được chuẩn hóa (cắt khoảng trắng, chữ thường) ở cả lúc đăng ký và lúc đăng nhập. Ràng buộc `UNIQUE` trong database là chốt chặn cuối khi hai người đăng ký cùng username trong cùng một thời điểm.

```
package com.fptaptech.auth.user;

public class UsernameAlreadyExistsException extends RuntimeException {
    public UsernameAlreadyExistsException(String username) {
        super("Username đã tồn tại: " + username);
    }
}
```

```
package com.fptaptech.auth.user;

import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.Locale;
```

```
@Service
public class UserService {

    private final UserRepository users;
    private final RoleRepository roles;
    private final PasswordEncoder passwordEncoder;

    public UserService(UserRepository users, RoleRepository roles,
PasswordEncoder passwordEncoder) {
        this.users = users;
        this.roles = roles;
        this.passwordEncoder = passwordEncoder;
    }

    public static String normalize(String username) {
        return username == null ? "" :
username.trim().toLowerCase(Locale.ROOT);
    }

    @Transactional
    public AppUser register(String rawUsername, String rawPassword) {
        String username = normalize(rawUsername);
        if (users.existsByUsername(username)) {
            throw new UsernameAlreadyExistsException(username);
        }
        Role userRole = roles.findByName(Role.USER)
                .orElseThrow(() -> new IllegalStateException("Thiếu dữ liệu
vai trò ROLE_USER"));

        AppUser user = new AppUser(username,
passwordEncoder.encode(rawPassword));
        user.getRoles().add(userRole);
        try {
            return users.saveAndFlush(user);
        } catch (DataIntegrityViolationException e) {
            // hai request đăng ký trùng username cùng lúc
            throw new UsernameAlreadyExistsException(username);
        }
    }
}
```

6.2 `UserDetailsService`

Spring Security gọi `loadUserByUsername` khi xử lý `POST /login`. Khi có đúng một bean `UserDetailsService` và một bean `PasswordEncoder`, Spring Security tự tạo `DaoAuthenticationProvider` — không cần khai báo `AuthenticationManager`.

```
package com.fptaptech.auth.security;

import com.fptaptech.auth.user.AppUser;
import com.fptaptech.auth.user.UserRepository;
import com.fptaptech.auth.user.UserService;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class DatabaseUserDetailsService implements UserDetailsService {

    private final UserRepository users;

    public DatabaseUserDetailsService(UserRepository users) {
        this.users = users;
    }

    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String username) {
        AppUser user = users.findByUsername(UserService.normalize(username))
                .orElseThrow(() -> new UsernameNotFoundException("Không tìm
thấy tài khoản"));

        return User.withUsername(user.getUsername())
                .password(user.getPasswordHash())
                .disabled(!user.isEnabled())
                .authorities(user.getRoles().stream()
                        .map(role -> new
SimpleGrantedAuthority(role.getName()))
```

```
                        .toList())
                .build();
    }
}
```

6.3 Tự đăng nhập sau khi đăng ký

Từ Spring Security 6, đặt `Authentication` vào `SecurityContextHolder` là chưa đủ: phải lưu `SecurityContext` qua `SecurityContextRepository`, nếu không request kế tiếp vẫn là ẩn danh. Đổi session id trước khi lưu để chống session fixation.

```
package com.fptaptech.auth.security;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationTok
en;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import
org.springframework.security.core.context.SecurityContextHolderStrategy;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.web.context.HttpSessionSecurityContextRepository
;
import org.springframework.security.web.context.SecurityContextRepository;
import org.springframework.stereotype.Component;

@Component
public class AutoLogin {

    private final UserDetailsService userDetailsService;
    private final SecurityContextRepository contextRepository = new
HttpSessionSecurityContextRepository();
    private final SecurityContextHolderStrategy holder =
SecurityContextHolder.getContextHolderStrategy();

    public AutoLogin(UserDetailsService userDetailsService) {
        this.userDetailsService = userDetailsService;
    }
```

```
    public void login(String username, HttpServletRequest request,
HttpServletResponse response) {
        UserDetails user = userDetailsService.loadUserByUsername(username);
        var authentication =
UsernamePasswordAuthenticationToken.authenticated(
                user, null, user.getAuthorities());

        if (request.getSession(false) != null) {
            request.changeSessionId();
        }

        SecurityContext context = holder.createEmptyContext();
        context.setAuthentication(authentication);
        holder.setContext(context);
        contextRepository.saveContext(context, request, response);
    }
}
```

7. Cấu hình Spring Security

Toàn bộ quy tắc bảo mật năm trong một bean `SecurityFilterChain`. Spring Security 7 chỉ còn lambda DSL (không còn `.and()`), và `POST /login`, `POST /logout` do framework xử lý — không viết controller cho hai endpoint này.

```
package com.fptaptech.auth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSec
urity;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

```
    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/registration", "/login",
"/error").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/", false)
                .permitAll())
            .logout(logout -> logout
                .logoutSuccessUrl("/login?logout")
                .permitAll())
            .headers(headers -> headers
                .contentSecurityPolicy(csp -> csp.policyDirectives(
                    "default-src 'self'; style-src 'self'
https://cdn.jsdelivr.net; img-src 'self' data:")));
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        // mặc định BCrypt, lưu dạng {bcrypt}$2a$10$...
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }
}
```

Cấu hình	Ý nghĩa
`requestMatchers(...).permitAll()`	Trang đăng ký, đăng nhập và trang lỗi ai cũng vào được; thiếu `/error` sẽ bị redirect vòng lặp khi có lỗi
`hasRole("ADMIN")`	So khớp authority `ROLE_ADMIN` (Spring tự thêm tiền tố `ROLE_`)
`anyRequest().authenticated()`	Mọi URL còn lại bắt buộc đăng nhập; người chưa đăng nhập bị chuyển về `/login`
`loginPage("/login")`	Dùng trang đăng nhập tự làm; form phải `POST /login` với trường `username`, `password`

Cấu hình	Ý nghĩa
`defaultSuccessUrl("/", false)`	Đăng nhập xong quay lại trang đang định vào, nếu không có thì về `/`
`logoutSuccessUrl("/login?logout")`	Đăng xuất bằng `POST /logout` (có CSRF token), xong chuyển về trang đăng nhập
`contentSecurityPolicy(...)`	Chỉ cho tải CSS từ chính site và CDN Bootstrap, chặn script lạ
CSRF	Bật mặc định; Thymeleaf tự chèn token vào mọi form có `th:action`

8. Controller và validation

Form đăng ký kiểm tra 3 tầng: ràng buộc từng trường (độ dài, ký tự cho phép), ràng buộc cấp lớp (hai mật khẩu khớp nhau), và kiểm tra nghiệp vụ trong service (username chưa tồn tại).

8.1 Form object

Dùng class có getter/setter thay vì `record` để Thymeleaf `th:field` đọc và ghi giá trị trực tiếp.

```
package com.fptaptech.auth.web;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

@PasswordsMatch
public class RegistrationForm {

    @NotBlank(message = "{registration.required}")
    @Size(min = 4, max = 32, message = "{registration.username.size}")
    @Pattern(regexp = "^[A-Za-z0-9._-]*$", message = "
{registration.username.pattern}")
    private String username;

    @NotBlank(message = "{registration.required}")
    @Size(min = 8, max = 64, message = "{registration.password.size}")
```

```
    private String password;

    @NotBlank(message = "{registration.required}")
    private String passwordConfirm;

    /** Không trả mật khẩu về trình duyệt khi form có lỗi. */
    public void clearPasswords() {
        this.password = "";
        this.passwordConfirm = "";
    }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getPasswordConfirm() { return passwordConfirm; }
    public void setPasswordConfirm(String passwordConfirm) {
this.passwordConfirm = passwordConfirm; }
}
```

8.2 Ràng buộc cấp lớp `@PasswordsMatch`

Lỗi được gắn vào trường `passwordConfirm` để hiển thị ngay dưới ô nhập lại mật khẩu.

```
package com.fptaptech.auth.web;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;

import java.lang.annotation.*;

@Documented
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = PasswordsMatchValidator.class)
public @interface PasswordsMatch {
    String message() default "{registration.passwords.mismatch}";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

```
package com.fptaptech.auth.web;

import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;

import java.util.Objects;

public class PasswordsMatchValidator implements
ConstraintValidator<PasswordsMatch, RegistrationForm> {

    @Override
    public boolean isValid(RegistrationForm form, ConstraintValidatorContext
context) {
        if (form.getPassword() == null || form.getPasswordConfirm() == null)
{
            return true; // để @NotBlank báo lỗi
        }
        if (Objects.equals(form.getPassword(), form.getPasswordConfirm())) {
            return true;
        }
        context.disableDefaultConstraintViolation();

context.buildConstraintViolationWithTemplate(context.getDefaultConstraintMess
ageTemplate())
                .addPropertyNode("passwordConfirm")
                .addConstraintViolation();
        return false;
    }
}
```

8.3 Thông báo lỗi tiếng Việt

`src/main/resources/messages.properties` (Spring Boot đọc file này bằng UTF-8 và dùng nó cho cả Bean Validation):

```
registration.required=Không được để trống
registration.username.size=Username phải dài từ {min} đến {max} ký tự
registration.username.pattern=Username chỉ gồm chữ cái, chữ số và các ký tự .
_ -
registration.password.size=Mật khẩu phải dài từ {min} đến {max} ký tự
registration.passwords.mismatch=Mật khẩu nhập lại không khớp
registration.username.duplicate=Username này đã được sử dụng
```

8.4 Controller

```
package com.fptaptech.auth.web;

import com.fptaptech.auth.security.AutoLogin;
import com.fptaptech.auth.user.UserService;
import com.fptaptech.auth.user.UsernameAlreadyExistsException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.validation.Valid;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class AuthController {

    private final UserService userService;
    private final AutoLogin autoLogin;

    public AuthController(UserService userService, AutoLogin autoLogin) {
        this.userService = userService;
        this.autoLogin = autoLogin;
    }

    @GetMapping("/registration")
    String registrationForm(Authentication authentication, Model model) {
        if (authentication != null) {
            return "redirect:/";
        }
        model.addAttribute("form", new RegistrationForm());
        return "registration";
    }

    @PostMapping("/registration")
    String register(@Valid @ModelAttribute("form") RegistrationForm form,
BindingResult result,
                    HttpServletRequest request, HttpServletResponse response)
{
```

```
        if (result.hasErrors()) {
            form.clearPasswords();
            return "registration";
        }
        try {
            userService.register(form.getUsername(), form.getPassword());
        } catch (UsernameAlreadyExistsException e) {
            result.rejectValue("username",
"registration.username.duplicate");
            form.clearPasswords();
            return "registration";
        }
        autoLogin.login(form.getUsername(), request, response);
        return "redirect:/";
    }

    @GetMapping("/login")
    String login(Authentication authentication) {
        return authentication != null ? "redirect:/" : "login";
    }

    @GetMapping("/")
    String welcome() {
        return "welcome";
    }
}
```

Tham số `Authentication` là `null` khi người dùng chưa đăng nhập, nên dùng được để chuyển hướng người đã đăng nhập ra khỏi trang đăng ký và đăng nhập.

9. Giao diện Thymeleaf

Ba trang dùng chung phần `<head>` nạp Bootstrap 5 từ CDN. Không dùng thuộc tính `style` inline vì CSP ở mục 7 chặn style inline; mọi định dạng đi qua class của Bootstrap.

9.1 `templates/fragments.html`

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:fragment="head(title)">
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title th:text="${title}">Auth</title>
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css
">
</head>
</html>
```

9.2 `templates/login.html`

Form gửi `POST /login` với đúng tên trường `username` và `password`. Spring Security chuyển về `/login?error` khi sai thông tin, `/login?logout` sau khi đăng xuất.

```
<!DOCTYPE html>
<html lang="vi" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments :: head('Đăng nhập')}"></head>
<body class="bg-light">
<main class="container py-5">
    <div class="row justify-content-center">
        <div class="col-md-5">
            <h1 class="h3 mb-4 text-center">Đăng nhập</h1>

            <div th:if="${param.error}" class="alert alert-danger">Sai
username hoặc mật khẩu.</div>
            <div th:if="${param.logout}" class="alert alert-success">Bạn đã
đăng xuất.</div>

            <form th:action="@{/login}" method="post" class="card card-body">
                <div class="mb-3">
                    <label for="username" class="form-label">Username</label>
                    <input id="username" name="username" class="form-control"
                           autocomplete="username" required autofocus>
                </div>
                <div class="mb-3">
                    <label for="password" class="form-label">Mật khẩu</label>
                    <input id="password" name="password" type="password"
```

```
class="form-control"
                            autocomplete="current-password" required>
                </div>
                <button type="submit" class="btn btn-primary w-100">Đăng
nhập</button>
            </form>

            <p class="text-center mt-3">Chưa có tài khoản? <a
th:href="@{/registration}">Đăng ký</a></p>
        </div>
    </div>
</main>
</body>
</html>
```

9.3 `templates/registration.html`

`th:object` gắn form với `RegistrationForm`; `th:errors` hiển thị thông báo lỗi tiếng Việt từ mục 8.3 ngay dưới từng ô.

```
<!DOCTYPE html>
<html lang="vi" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments :: head('Đăng ký')}"></head>
<body class="bg-light">
<main class="container py-5">
    <div class="row justify-content-center">
        <div class="col-md-5">
            <h1 class="h3 mb-4 text-center">Tạo tài khoản</h1>

            <form th:action="@{/registration}" th:object="${form}"
method="post" class="card card-body" novalidate>
                <div class="mb-3">
                    <label for="username" class="form-label">Username</label>
                    <input th:field="*{username}" class="form-control"
autocomplete="username"
                           th:classappend="${#fields.hasErrors('username')} ?
'is-invalid'">
                    <div class="invalid-feedback" th:errors="*{username}">
</div>
                </div>
                <div class="mb-3">
                    <label for="password" class="form-label">Mật khẩu</label>
```

```
                    <input th:field="*{password}" type="password"
class="form-control" autocomplete="new-password"
                           th:classappend="${#fields.hasErrors('password')} ?
'is-invalid'">
                    <div class="invalid-feedback" th:errors="*{password}">
</div>
                </div>
                <div class="mb-3">
                    <label for="passwordConfirm" class="form-label">Nhập lại
mật khẩu</label>
                    <input th:field="*{passwordConfirm}" type="password"
class="form-control" autocomplete="new-password"

th:classappend="${#fields.hasErrors('passwordConfirm')} ? 'is-invalid'">
                    <div class="invalid-feedback" th:errors="*
{passwordConfirm}"></div>
                </div>
                <button type="submit" class="btn btn-primary w-100">Đăng
ký</button>
            </form>

            <p class="text-center mt-3">Đã có tài khoản? <a
th:href="@{/login}">Đăng nhập</a></p>
        </div>
    </div>
</main>
</body>
</html>
```

9.4 `templates/welcome.html`

Dialect `sec:` (từ `thymeleaf-extras-springsecurity6`, Spring Boot tự đăng ký) đọc thông tin người đang đăng nhập và ẩn/hiện nội dung theo vai trò. Đăng xuất phải là form `POST` để có CSRF token.

```
<!DOCTYPE html>
<html lang="vi" xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head th:replace="~{fragments :: head('Trang chủ')}"></head>
<body class="bg-light">
<main class="container py-5">
    <h1 class="h3">Xin chào, <span sec:authentication="name">user</span>!
</h1>
    <p class="text-secondary">Vai trò: <span
sec:authentication="principal.authorities">[ROLE_USER]</span></p>

    <p sec:authorize="hasRole('ADMIN')">
        <a th:href="@{/admin}" class="btn btn-warning">Vào khu quản trị</a>
    </p>

    <form th:action="@{/logout}" method="post">
        <button type="submit" class="btn btn-outline-secondary">Đăng
xuất</button>
    </form>
</main>
</body>
</html>
```

10. Chạy và kiểm thử

10.1 Cấu hình

`src/main/resources/application.yaml` — không khai báo `spring.datasource.*` vì Docker Compose support tự sinh từ `compose.yaml`:

```
spring:
  application:
    name: auth
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate       # schema do Flyway quản lý
  thymeleaf:
    cache: false               # dev; bật lại trên production

server:
  servlet:
    session:
      timeout: 30m
      cookie:
        http-only: true
        same-site: lax
```

`compose.yaml` ở thư mục gốc project:

```
services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: auth
      MYSQL_USER: auth
      MYSQL_PASSWORD: auth_pw
      MYSQL_ROOT_PASSWORD: root_pw
    ports:
      - "3306"
```

10.2 Chạy ứng dụng

```
./mvnw spring-boot:run
# Log: Using Docker Compose file .../compose.yaml
#      Successfully applied 1 migration to schema `auth`
#      Started AuthApplication
```

10.3 Kiểm thử thủ công

1. Mở `http://localhost:8080` → bị chuyển về `/login` vì chưa đăng nhập.

2. Bấm **Đăng ký**, gửi form trống → mỗi ô hiện lỗi "Không được để trống".
3. Nhập mật khẩu và mật khẩu nhập lại khác nhau → lỗi "Mật khẩu nhập lại không khớp" dưới ô thứ hai.
4. Đăng ký `Alice` / `Secret#2026` → vào thẳng trang chào mừng với tên `alice`, vai trò `[ROLE_USER]`.
5. Đăng ký lại `ALICE` ở cửa sổ ẩn danh → lỗi "Username này đã được sử dụng".
6. Bấm **Đăng xuất** → về `/login?logout`; đăng nhập sai mật khẩu → `/login?error`.
7. Truy cập `/admin` bằng tài khoản `alice` → `403 Forbidden`.
8. Kiểm tra database: `SELECT username, password_hash FROM users;` → mật khẩu có dạng `{bcrypt}$2a$10$...`.

10.4 Test tự động

`src/test/resources/application-test.yaml` dùng H2 ở chế độ tương thích MySQL, nên migration Flyway chạy được không cần Docker:

```
spring:
  datasource:
    url:
jdbc:h2:mem:authtest;MODE=MySQL;DATABASE_TO_LOWER=TRUE;DB_CLOSE_DELAY=-1
  docker:
    compose:
      enabled: false
```

```
package com.fptaptech.auth;

import com.fptaptech.auth.user.UserService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import
org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.web.servlet.MockMvc;

import static
org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestB
uilders.formLogin;
import static
```

```
org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestP
ostProcessors.csrf;
import static
org.springframework.security.test.web.servlet.response.SecurityMockMvcResultM
atchers.authenticated;
import static
org.springframework.security.test.web.servlet.response.SecurityMockMvcResultM
atchers.unauthenticated;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class AuthFlowTest {

    private static final String PASSWORD = "Secret#2026";

    @Autowired MockMvc mvc;
    @Autowired UserService userService;

    @Test
    void anonymousUserIsRedirectedToLogin() throws Exception {
        mvc.perform(get("/"))
           .andExpect(status().is3xxRedirection())
           .andExpect(redirectedUrlPattern("**/login"));
    }

    @Test
    void registrationLogsUserInAutomatically() throws Exception {
        mvc.perform(post("/registration").with(csrf())
                .param("username", "Alice")
                .param("password", PASSWORD)
                .param("passwordConfirm", PASSWORD))
           .andExpect(redirectedUrl("/"))

.andExpect(authenticated().withUsername("alice").withRoles("USER"));
    }
```

```
    @Test
    void duplicateUsernameIsRejected() throws Exception {
        userService.register("bob", PASSWORD);
        mvc.perform(post("/registration").with(csrf())
                .param("username", "BOB")
                .param("password", PASSWORD)
                .param("passwordConfirm", PASSWORD))
           .andExpect(status().isOk())
           .andExpect(model().attributeHasFieldErrorCode("form", "username",
"registration.username.duplicate"))
           .andExpect(unauthenticated());
    }

    @Test
    void mismatchedPasswordsAreRejected() throws Exception {
        mvc.perform(post("/registration").with(csrf())
                .param("username", "dave")
                .param("password", PASSWORD)
                .param("passwordConfirm", "Other#2026"))
           .andExpect(status().isOk())
           .andExpect(model().attributeHasFieldErrors("form",
"passwordConfirm"));
    }

    @Test
    void formLoginWithValidCredentialsSucceeds() throws Exception {
        userService.register("carol", PASSWORD);
        mvc.perform(formLogin("/login").user("carol").password(PASSWORD))
           .andExpect(authenticated().withUsername("carol"));
    }

    @Test
    void wrongPasswordRedirectsToLoginError() throws Exception {
        userService.register("erin", PASSWORD);
        mvc.perform(formLogin("/login").user("erin").password("wrong-
password"))
           .andExpect(redirectedUrl("/login?error"))
           .andExpect(unauthenticated());
    }

    @Test
    void postWithoutCsrfTokenIsForbidden() throws Exception {
        mvc.perform(post("/registration").param("username", "frank"))
```

```
            .andExpect(status().isForbidden());
    }

    @Test
    @WithMockUser(roles = "USER")
    void regularUserCannotOpenAdminArea() throws Exception {
        mvc.perform(get("/admin"))
            .andExpect(status().isForbidden());
    }
}
```

Chạy: `./mvnw test`. Mỗi test dùng một username riêng vì các test dùng chung một database H2 trong cùng Spring context.

11. Bài tập ứng dụng: Cổng tài khoản học viên FPT Aptech

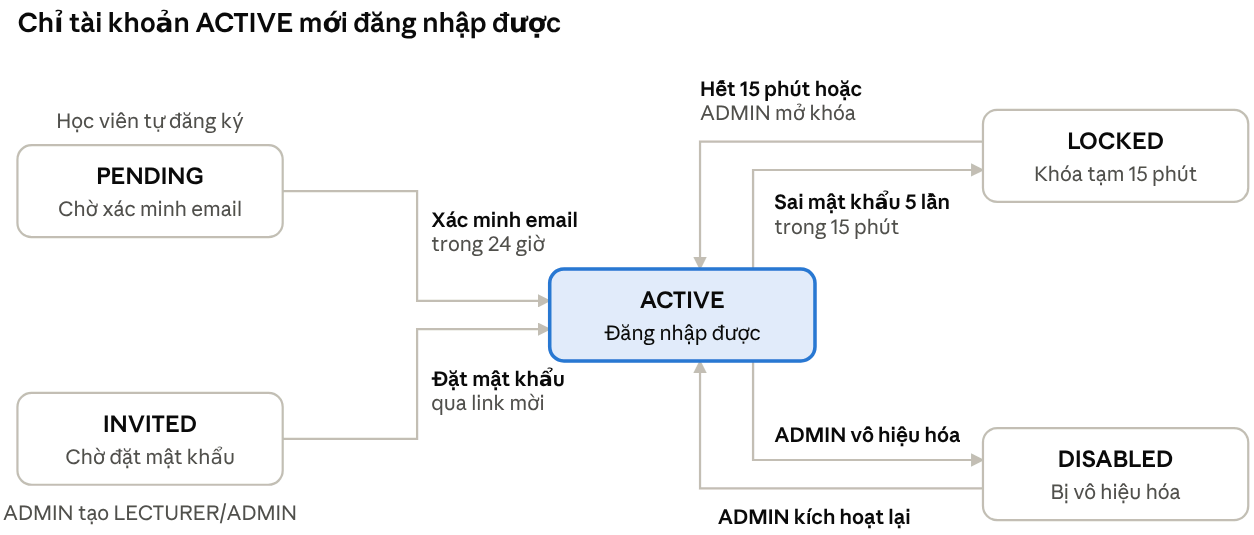
Mở rộng ứng dụng trong tutorial thành cổng tài khoản dùng chung cho học viên, giảng viên và quản trị viên. Ứng dụng phải đạt mức an toàn của một hệ thống thật: xác minh email, khôi phục mật khẩu, khóa tài khoản khi bị dò mật khẩu, xác thực 2 bước, quản lý phiên và nhật ký kiểm toán.

11.1 Vai trò

Vai trò	Được làm	Xác thực 2 bước
`STUDENT`	Tự đăng ký; xem, sửa hồ sơ của mình; đổi mật khẩu; quản lý phiên đăng nhập của mình	Tùy chọn
`LECTURER`	Như STUDENT; xem danh sách học viên (chỉ đọc)	Bắt buộc
`ADMIN`	Quản lý mọi tài khoản: tìm kiếm, khóa/mở khóa, gán vai trò, buộc đăng xuất, xem nhật ký	Bắt buộc

Tài khoản `LECTURER` và `ADMIN` không tự đăng ký được: chỉ ADMIN tạo, hệ thống gửi email mời đặt mật khẩu.

11.2 Vòng đời tài khoản



vòng đời tài khoản · 5 trạng thái

Tài khoản ở mọi trạng thái khác `ACTIVE` đăng nhập đều thất bại với cùng một thông báo chung. Chuyển `ACTIVE` → `DISABLED` phải hủy ngay mọi phiên và token ghi nhớ đăng nhập của tài khoản đó.

11.3 Yêu cầu chức năng

Mã	Chức năng	Tiêu chí nghiệm thu
F1	Đăng ký băng email	Tài khoản mới ở trạng thái `PENDING`, chưa đăng nhập được; hệ thống gửi email chứa link xác minh
F2	Xác minh email	Token dùng một lần, hết hạn sau 24 giờ, lưu dạng băm SHA-256 trong database; gửi lại link tối đa 3 lần mỗi giờ
F3	Quên mật khẩu	Token hết hạn sau 30 phút, dùng một lần; phản hồi giống hệt nhau dù email có tồn tại hay không; đặt lại xong thì mọi phiên cũ bị hủy
F4	Chính sách mật khẩu	Tối thiểu 12 ký tự; không chứa username/email; không năm trong danh sách 10.000 mật khẩu phổ biến; không trùng 5 mật khẩu gần nhất

Mã	Chức năng	Tiêu chí nghiệm thu
F5	Khóa tạm thời	Sai mật khẩu 5 lần trong 15 phút thì khóa 15 phút; thông báo lỗi không tiết lộ tài khoản có tồn tại hay không
F6	Xác thực 2 bước (TOTP, RFC 6238)	Bật bằng QR code; có 10 mã khôi phục dùng một lần; LECTURER/ADMIN chưa bật thì sau khi đăng nhập bị buộc vào trang thiết lập
F7	Ghi nhớ đăng nhập	Remember-me lưu token trong database, hiệu lực 14 ngày; tự thu hồi khi đổi mật khẩu
F8	Quản lý phiên	Tối đa 2 phiên đồng thời mỗi tài khoản; trang "Thiết bị đăng nhập" hiện thời điểm, IP, trình duyệt và cho đăng xuất từ xa từng phiên
F9	Đổi mật khẩu khi đang đăng nhập	Phải nhập mật khẩu hiện tại; xong thì mọi phiên khác bị hủy, phiên hiện tại giữ nguyên
F10	Quản trị tài khoản	Danh sách phân trang, lọc theo vai trò/trạng thái, tìm theo email; khóa/mở khóa, gán vai trò, buộc đăng xuất; hệ thống luôn còn ít nhất 1 ADMIN đang hoạt động và ADMIN không tự gỡ quyền ADMIN của mình
F11	Nhật ký kiểm toán	Ghi mọi sự kiện đăng nhập thành công/thất bại, đăng xuất, khóa/mở khóa, đổi mật khẩu, đổi vai trò, bật/tắt 2 bước; mỗi bản ghi có thời điểm, tài khoản, IP, user-agent; ADMIN xem và lọc được

11.4 Yêu cầu bảo mật và phi chức năng

- Mật khẩu băm bằng `DelegatingPasswordEncoder`; tài khoản cũ dùng thuật toán yếu hơn tự được băm lại khi đăng nhập thành công.
- Bí mật TOTP mã hóa khi lưu (AES-GCM, khóa lấy từ biến môi trường), không lưu dạng rõ.
- Mọi form có CSRF; cookie phiên `HttpOnly`, `SameSite=Lax`, `Secure` trên production; bật HSTS và CSP trên production.
- Giới hạn tần suất theo IP: tối đa 10 lần đăng ký và 20 lần gửi email quên mật khẩu mỗi giờ.
- Không ghi mật khẩu, token hay mã TOTP vào log.

- Thời gian phản hồi của `POST /login` khi username không tồn tại và khi sai mật khẩu chênh nhau không quá 50 ms (chống dò username theo thời gian).

11.5 Ràng buộc kỹ thuật

Spring Boot 4.1.x, JDK 25, Spring Security 7.1, Thymeleaf, Spring Data JPA, MySQL 8.4, Flyway (mỗi thay đổi schema là một migration mới, không sửa `V1`), Spring Session JDBC cho F8. Gửi email qua SMTP; môi trường dev dùng Mailpit khai báo trong `compose.yaml`. Test tích hợp dùng `spring-security-test`.

11.6 Sản phẩm nộp

- Repository mã nguồn với `README` hướng dẫn chạy bằng một lệnh `./mvnw spring-boot:run`.
- Bộ test tự động phủ các luồng F1–F11, gồm cả trường hợp lỗi: token hết hạn, token dùng lại, khóa tài khoản, sai mã TOTP, xóa ADMIN cuối cùng.
- Tài liệu 1–2 trang mô tả mô hình mối đe dọa (threat model): các kiểu tấn công đã tính tới và cách hệ thống chặn từng kiểu.